



COMPUTACION GRAFICAS Y SUS
APLICACIONES – COM3201-1

Algoritmos de ráster y Triangulación

COM3201-1
Profesores:
Alfonso Ehijo
Ignacio Bugueño

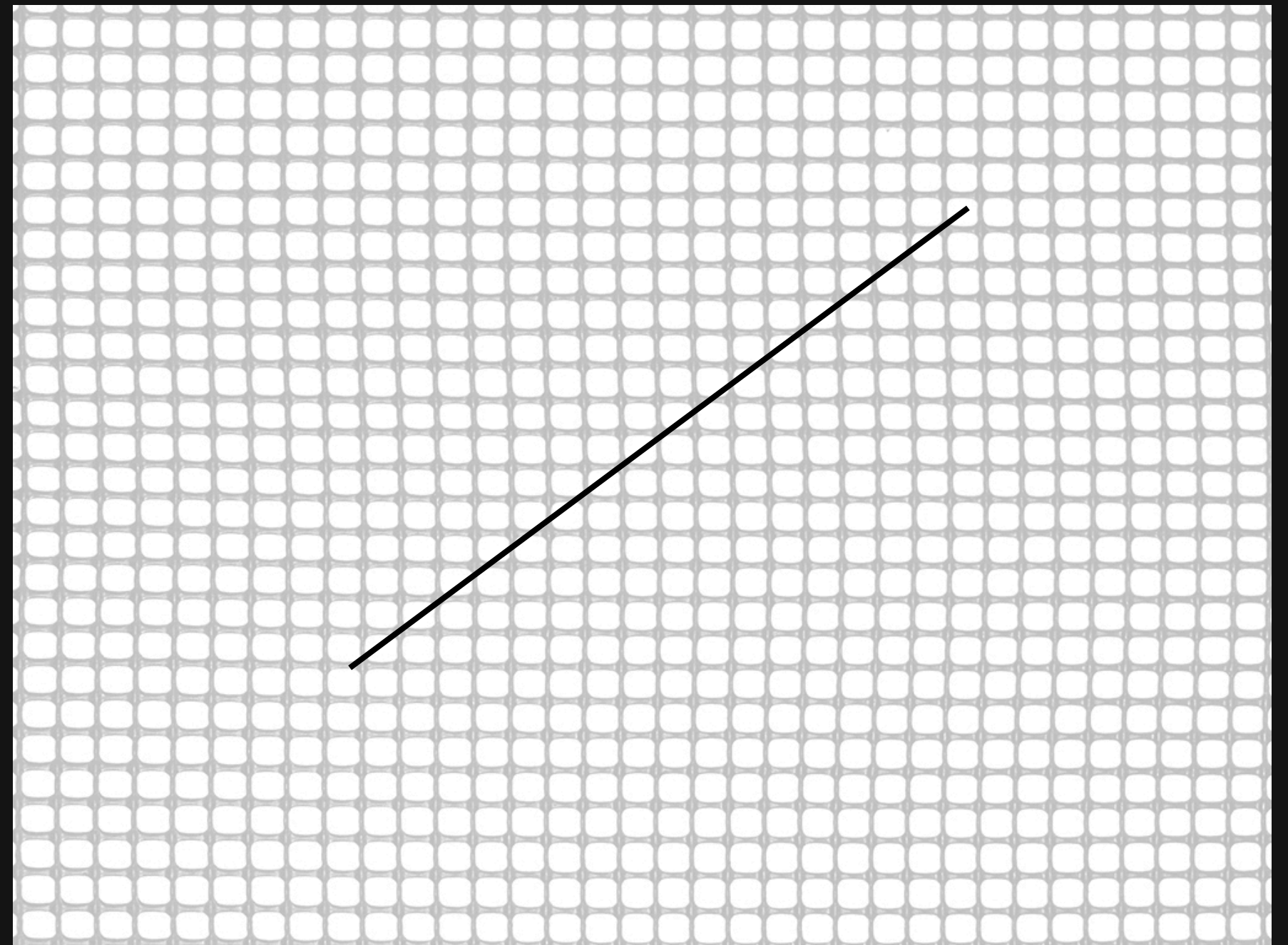
Ayudante
Alan E. Molina H.
alan.molina@pregrado.uoh.cl

ALGORITMOS DE RÁSTER



SEGMENTO DE LINEA

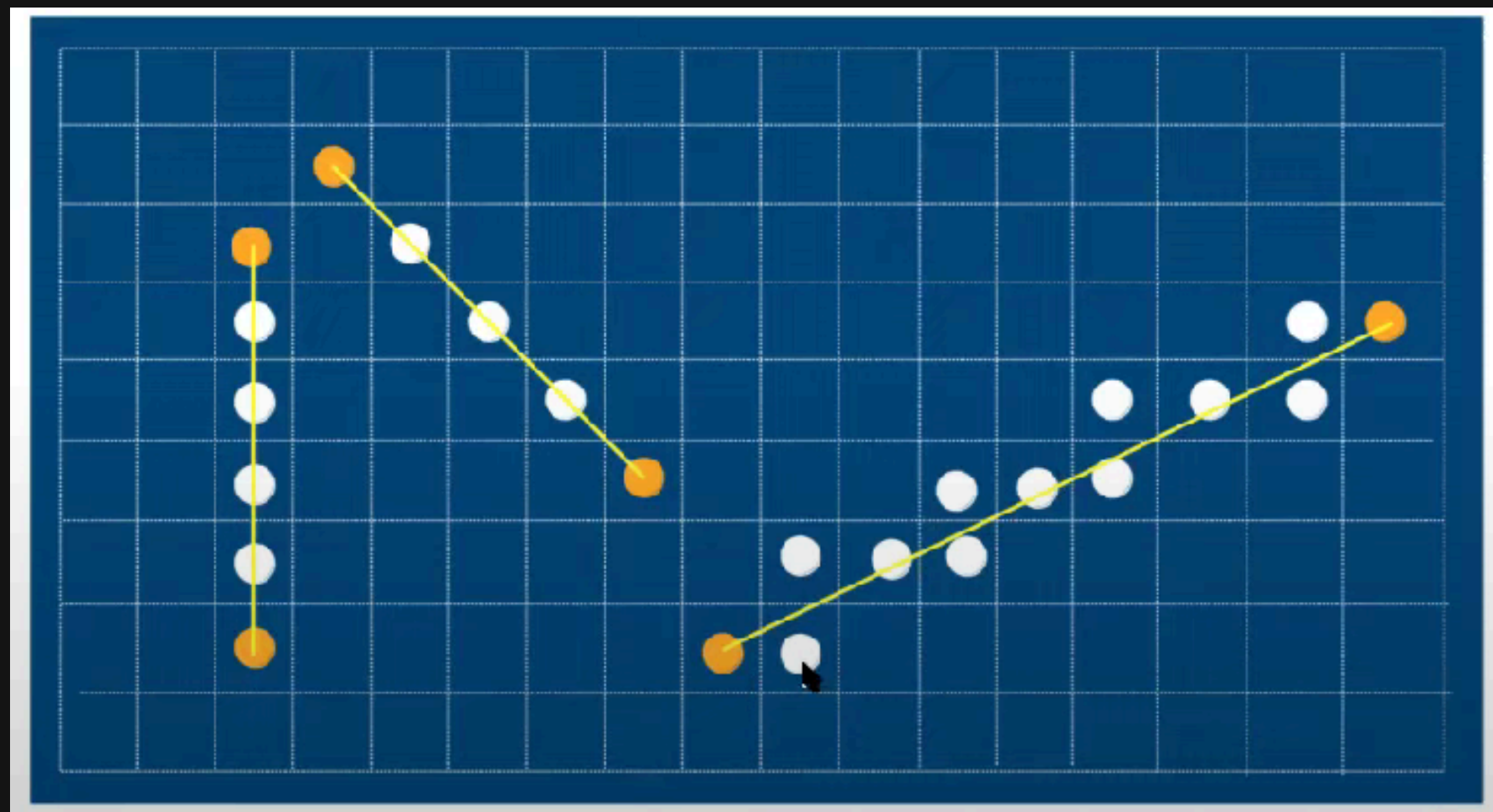
- + Arreglo 2D de pixles
- + Píxeles centrados en coordenadas enteras
- + `setPixel(x,y)`: funcion para iluminar un pixel



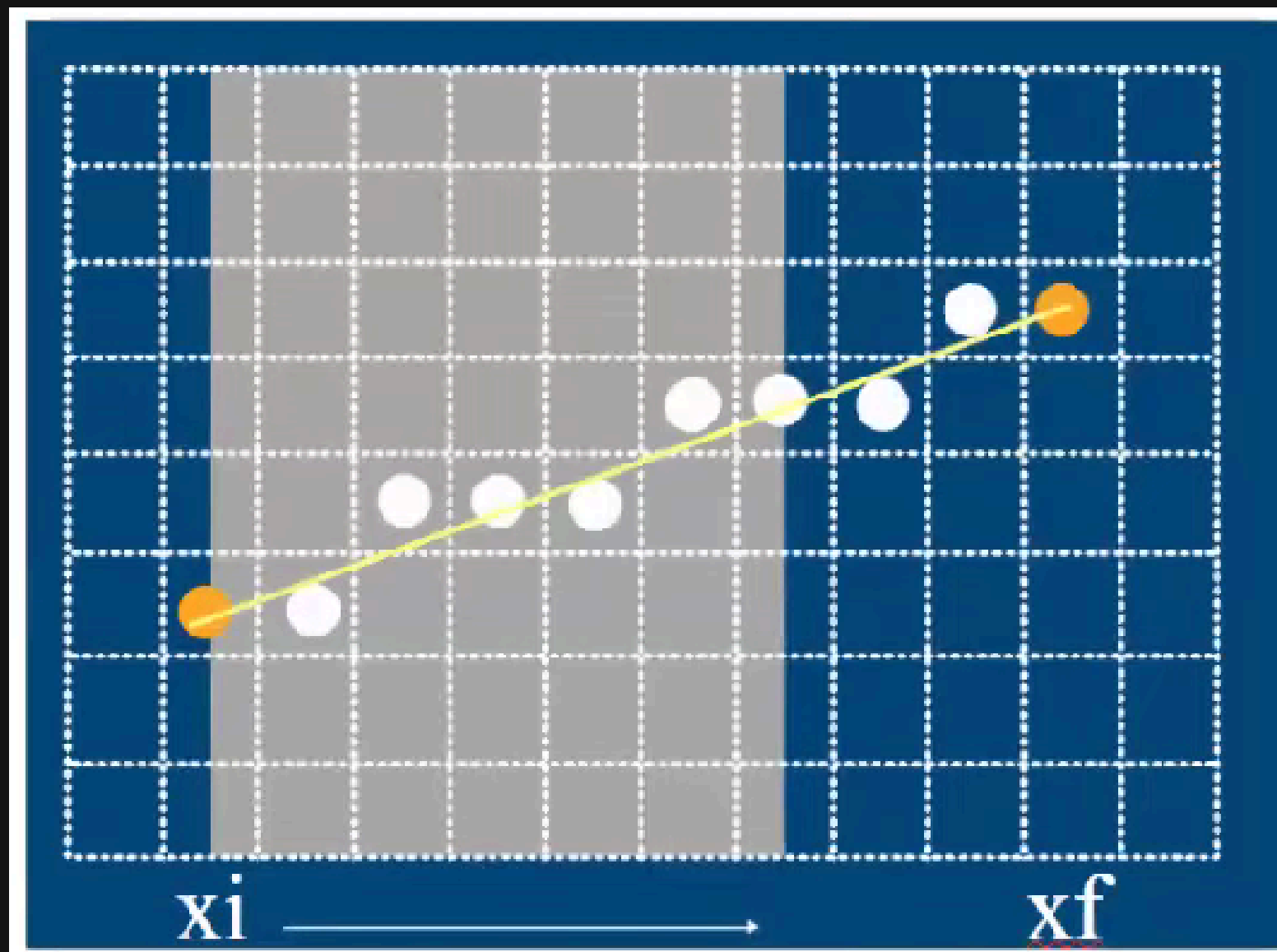


SEGMENTO DE LINEA

Dado los puntos extremos de un segmento: ¿Como podemos identificar los pixeles que lo representan?



ALGORITMO INGENUO



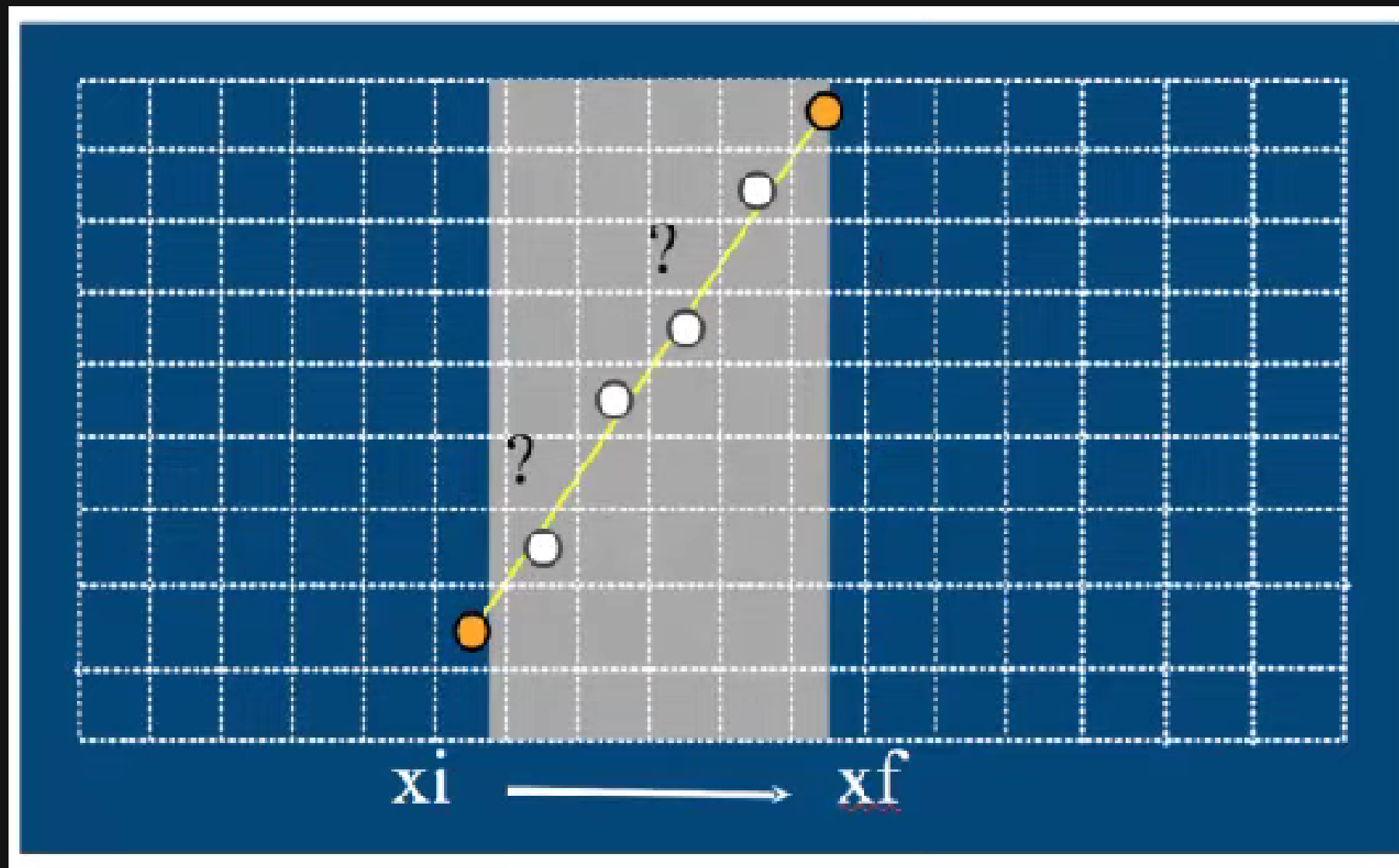
Algoritmo Ingenuo
Extremos : $(x_i, y_i) \rightarrow (x_f, y_f)$.

Sea f la función que describe
 $y=f(x)$.

todo x pertenece a $[x_i, x_f,$
 $\text{setPixel}(x, \text{round}(y))]$

¿Problema?

PROBLEMA

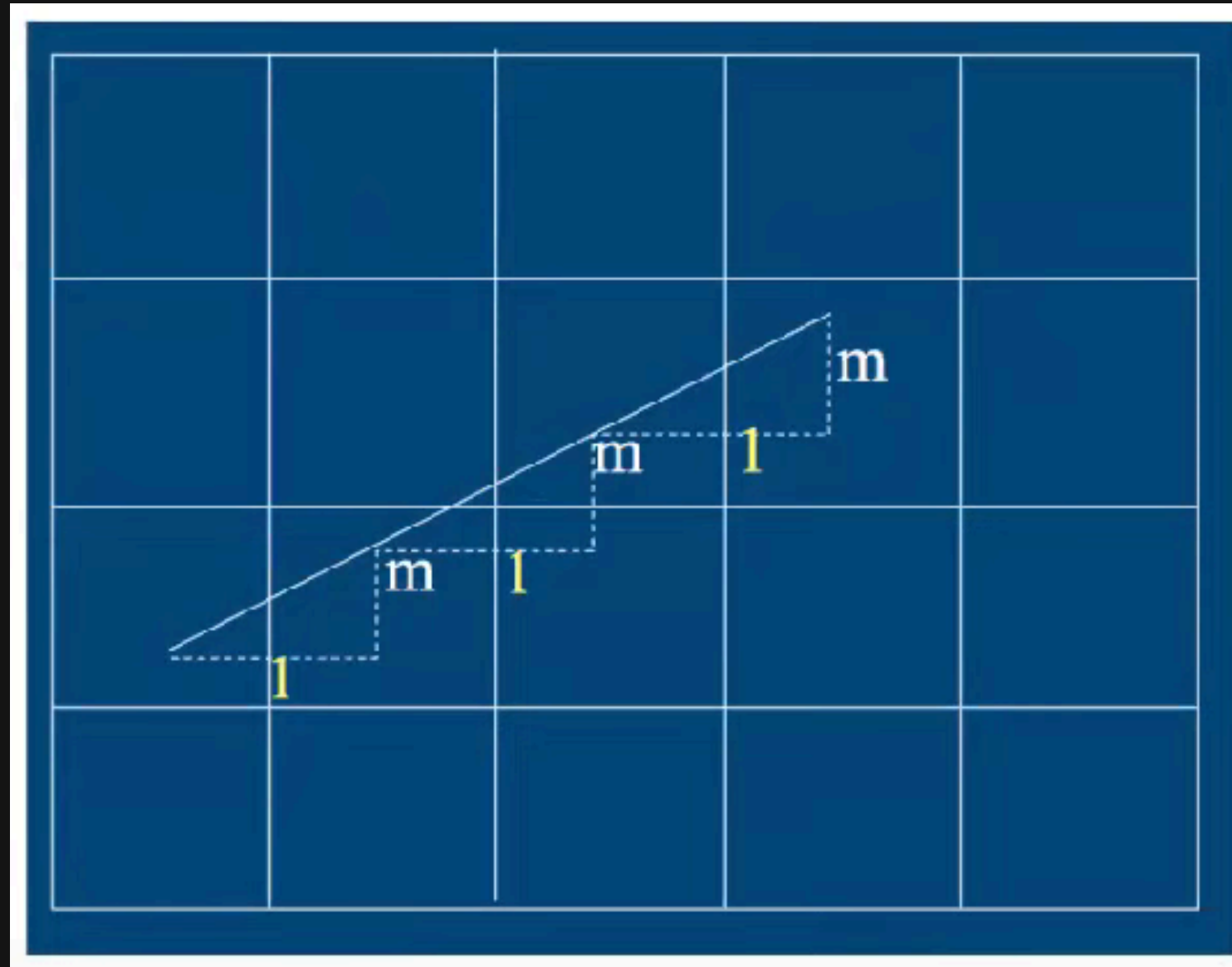


Si la pendiente $m > 1$ la línea se puede cortar.

$$f(x) = \frac{3}{2}x = y$$

x	1	2	3	4	5	6
y	1.5	3	4.5	6	7.5	9
$round(y)$	2	3	5	6	8	9

ALGORITMO INCREMENTAL



```
1:  $x \leftarrow x_i$ 
2:  $y \leftarrow y_i$ 
3:  $setPixel(x, round(y))$ 
4: while  $x < x_f$  do
5:    $x \leftarrow x + 1$ 
6:    $y \leftarrow y + m$ 
7:    $setPixel(x, round(y))$ 
8: end while
```

ALGORITMO INCREMENTAL

Si la pendiente $m > 1$ la línea se puede cortar.

$$f(x) = \frac{3}{2}x = y, x_i = 1, y_i = 1$$

x	1	2	3	4	5	6
y	1	2.5	4	5.5	7	8.5
$round(y)$	1	3	4	6	7	9

```
1:  $x \leftarrow x_i$ 
2:  $y \leftarrow y_i$ 
3:  $setPixel(x, round(y))$ 
4: while  $x < x_f$  do
5:    $x \leftarrow x + 1$ 
6:    $y \leftarrow y + m$ 
7:    $setPixel(x, round(y))$ 
8: end while
```

Algoritmo de Bresenham

Definición y Características

Def: Algoritmo eficiente para dibujar una línea en una pantalla raster (compuesta de pixles)

Características:

Usa aritmética simple.

Supone la pendiente m en el rango $[0,1]$.

Parte del punto inferior-izquierdo al punto superior derecho.

[Repositorio de algoritmos basados en brasenham](#)

Formalización

Ecuación de la recta:

$$y = f(x) = mx + c$$

Punto de la recta (valor exacto):

$$(x, f(x)) = (x, mx + c)$$

Punto de la recta (valor en pantalla):

$$(x, \text{round}(f(x))) = (x, \text{round}(mx + c))$$

Pero sabiendo que $\Delta x = 1$ en nuestro caso, obtenemos finalmente que:

$$y_2 = y_1 + m$$

$$x_2 = x_1 + 1$$

Dado que:

$$x_2 = x_1 + \Delta x$$

$$y_1 = mx_1 + c$$

$$y_2 = mx_2 + c$$

Luego:

$$y_2 = mx_2 + c$$

$$= m(x_1 + \Delta x) + c$$

$$= mx_1 + m\Delta x + c$$

$$= (mx_1 + c) + m\Delta x$$

$$= y_1 + m\Delta x$$

Formalización

- Segmento de recta desde (x_i, y_i) a (x_f, y_f) .
- (x, y) : centro de un pixel
- ε : error (distancia) a la ordenada (y) del pixel.
- $\varepsilon \in [-0,5, 0,5]$
- $y + \varepsilon$: valor exacto de la ordenada de la recta
- m : pendiente de la recta

La gran pregunta para la siguiente iteración $(x + 1)$ es: ¿Qué pixel pintamos? $(x + 1, y)$ o $(x + 1, y + 1)$

Respuesta: dependerá del error y notar que éste es inicialmente 0. En la segunda iteración será m . Cuando el error llegue a ser superior a 0.5 entonces debemos “dar el salto” y al actualizarlo éste se debe volver negativo.

Pseudo algoritmo 1

Pseudo algoritmo 1

```
1:  $\varepsilon \leftarrow 0$ 
2:  $y \leftarrow y_i$ 
3: for  $x = x_i; x < x_f; x++$  do
4:    $setPixel(x, y)$ 
5:   if  $\varepsilon + m < 0,5$  then
6:      $\varepsilon \leftarrow \varepsilon + m$ 
7:   else
8:      $y \leftarrow y + 1$ 
9:      $\varepsilon \leftarrow \varepsilon + m - 1$ 
10:  end if
11: end for
```

¿Mejoras?

Notar que m es float.

¿Se podrá hacer solo con álgebra entera?

Formalización

Dado que $m = \frac{\Delta y}{\Delta x} = \frac{y_f - y_i}{x_f - x_i}$

$$\varepsilon + m < 0,5$$

$$\varepsilon + \frac{\Delta y}{\Delta x} < 0,5$$

$$2\varepsilon\Delta x + 2\Delta y < \Delta x$$

$$2(\varepsilon' + \Delta y) < \Delta x$$

Para actualizar el error teníamos que
si $\varepsilon + m < 0,5$:

$$\varepsilon \leftarrow \varepsilon + m$$

$$\varepsilon\Delta x \leftarrow \varepsilon\Delta x + \Delta y$$

$$\varepsilon' \leftarrow \varepsilon' + \Delta y$$

Y en el caso contrario:

Para el último paso se utilizó $\varepsilon' = \varepsilon\Delta x$.

$$\varepsilon \leftarrow \varepsilon + m - 1$$

$$\varepsilon\Delta x \leftarrow \varepsilon\Delta x + \Delta y - \Delta x$$

$$\varepsilon' \leftarrow \varepsilon' + \Delta y - \Delta x$$

Pseudo algoritmo 1

Pseudo algoritmo 2

```
1:  $\varepsilon' \leftarrow 0$ 
2:  $y \leftarrow y_i$ 
3: for  $x = x_i; x < x_f; x++$  do
4:   setPixel( $x, y$ )
5:   if  $2(\varepsilon' + \Delta y) < \Delta x$  then
6:      $\varepsilon' \leftarrow \varepsilon' + \Delta y$ 
7:   else
8:      $y \leftarrow y + 1$ 
9:      $\varepsilon' \leftarrow \varepsilon' + \Delta y - \Delta x$ 
10:  end if
11: end for
```

!!!Solo valores enteros!!!

Código de ejemplo para generar visualizar líneas continuas

```
import pygame

def main():
    pygame.init()
    screen = pygame.display.set_mode((640, 480))
    clock = pygame.time.Clock()

    radius = 15
    x = 0
    y = 0
    mode = 'blue'
    points = []

    while True:

        pressed = pygame.key.get_pressed()

        alt_held = pressed[pygame.K_LALT] or pressed[pygame.K_RALT]
        ctrl_held = pressed[pygame.K_LCTRL] or pressed[pygame.K_RCTRL]

        for event in pygame.event.get():

            # determin if X was clicked, or Ctrl+W or Alt+F4 was used
            if event.type == pygame.QUIT:
                return
```

```
import pygame

def main():
    pygame.init()
    screen = pygame.display.set_mode((640, 480))
    clock = pygame.time.Clock()

    radius = 15
    x = 0
    y = 0
    mode = 'blue'
    points = []

    while True:

        pressed = pygame.key.get_pressed()

        alt_held = pressed[pygame.K_LALT] or pressed[pygame.K_RALT]
        ctrl_held = pressed[pygame.K_LCTRL] or pressed[pygame.K_RCTRL]

        for event in pygame.event.get():

            # determin if X was clicked, or Ctrl+W or Alt+F4 was used
            if event.type == pygame.QUIT:
                return
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_w and ctrl_held:
                    return
                if event.key == pygame.K_F4 and alt_held:
                    return
                if event.key == pygame.K_ESCAPE:
                    return

            # determine if a letter key was pressed
            if event.key == pygame.K_r:
                mode = 'red'
            elif event.key == pygame.K_g:
                mode = 'green'
```



```

        # determine if a letter key was pressed
        if event.key == pygame.K_r:
            mode = 'red'
        elif event.key == pygame.K_g:
            mode = 'green'
        elif event.key == pygame.K_b:
            mode = 'blue'

    if event.type == pygame.MOUSEBUTTONDOWN:
        if event.button == 1: # left click grows radius
            radius = min(200, radius + 1)
        elif event.button == 3: # right click shrinks radius
            radius = max(1, radius - 1)

    if event.type == pygame.MOUSEMOTION:
        # if mouse moved, add point to list
        position = event.pos
        points = points + [position]
        points = points[-256:]

    screen.fill((0, 0, 0))

    # draw all points
    i = 0
    while i < len(points) - 1:
        drawLineBetween(screen, i, points[i], points[i + 1], radius, mode)
        i += 1

    pygame.display.flip()

    clock.tick(60)

```

```

def drawLineBetween(screen, index, start, end, width, color_mode):
    c1 = max(0, min(255, 2 * index - 256))
    c2 = max(0, min(255, 2 * index))

```

```

def drawLineBetween(screen, index, start, end, width, color_mode):
    c1 = max(0, min(255, 2 * index - 256))
    c2 = max(0, min(255, 2 * index))

    if color_mode == 'blue':
        color = (c1, c1, c2)
    elif color_mode == 'red':
        color = (c2, c1, c1)
    elif color_mode == 'green':
        color = (c1, c2, c1)

    dx = start[0] - end[0]
    dy = start[1] - end[1]
    iterations = max(abs(dx), abs(dy))

    for i in range(iterations):
        progress = 1.0 * i / iterations
        aprogress = 1 - progress
        x = int(aprogress * start[0] + progress * end[0])
        y = int(aprogress * start[1] + progress * end[1])
        pygame.draw.circle(screen, color, (x, y), width)

```

```

main()

```

Ejercicio:

Se requiere implementar un código que pueda dibujar una línea continua mediante el movimiento de entrada por mouse y que pueda salir con tecla escape.

Hint: Use las librerías llamadas:
sys, pygame, time, funciones

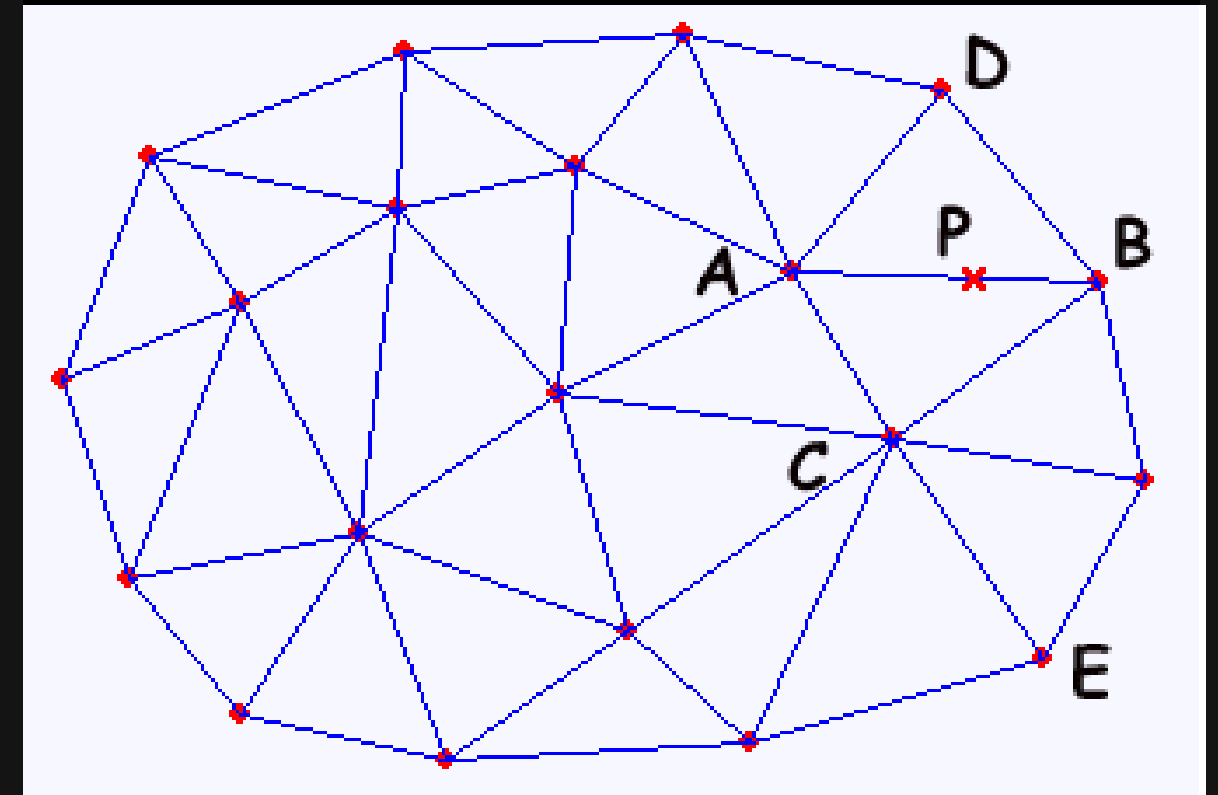
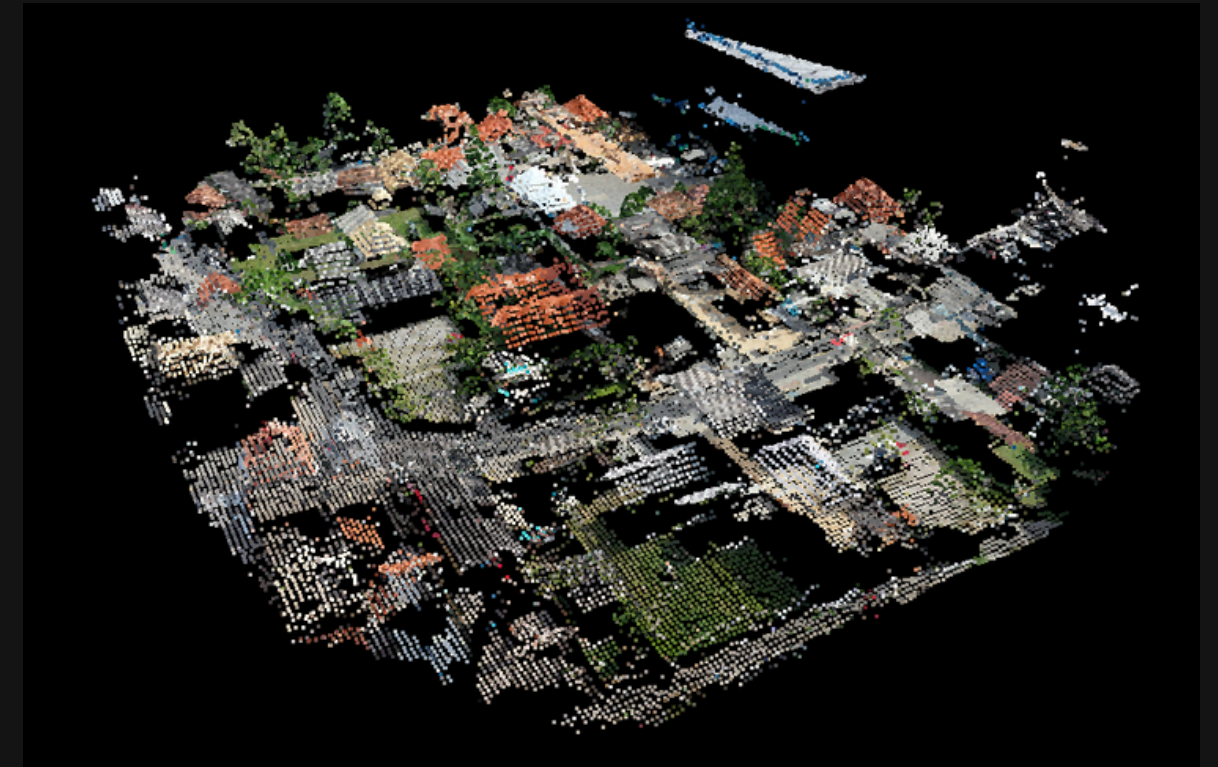
TRIANGULACIÓN

Conceptos previos

Las nubes de puntos

Se componen de un conjunto de vértices, elementos o puntos, que contienen la información necesaria para generar un modelo virtual. Es una representación espacial tridimensional que permite estructurar la información obtenida.

Se puede ver como una gran “montón” de puntos georreferenciados, capaz de recrear áreas y estructuras.

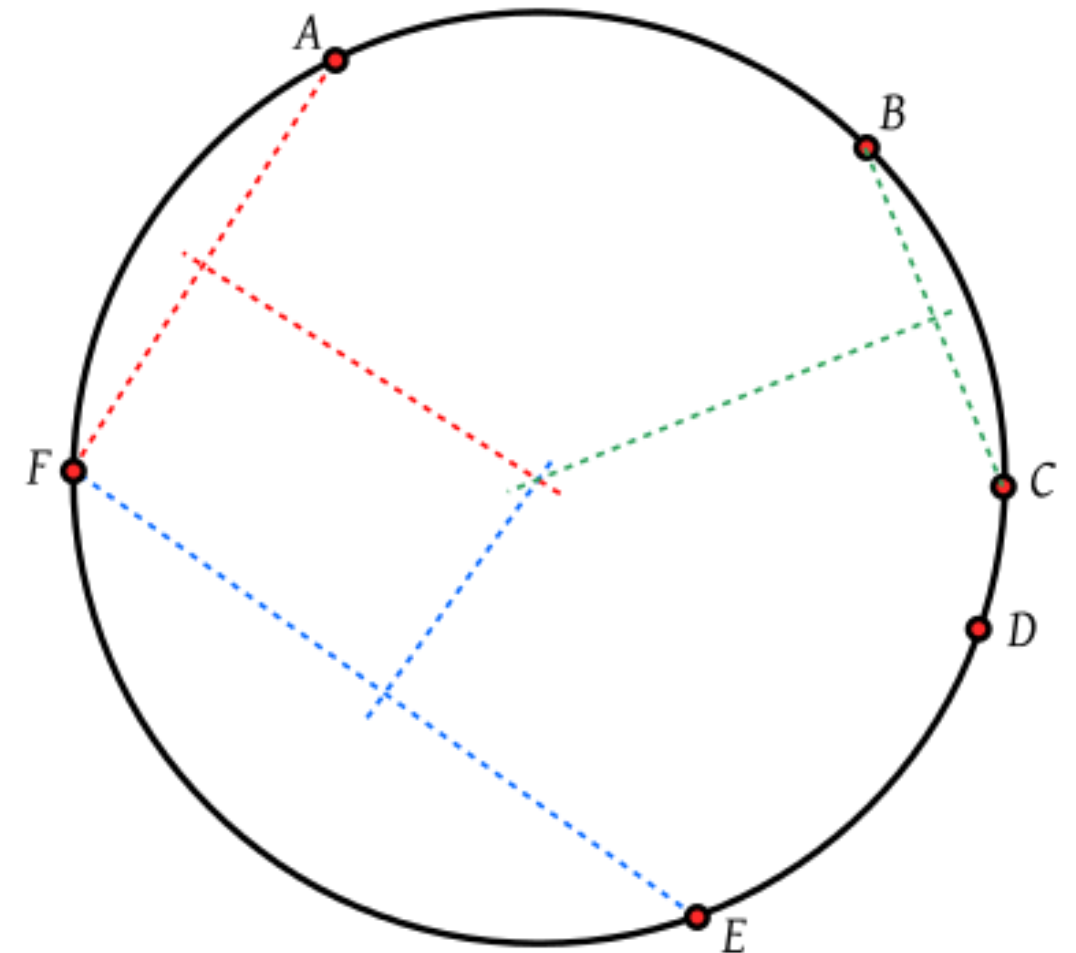


Conceptos previos

Puntos Concíclicos

Se dice que un conjunto de puntos es concíclicos (o cocíclico) si pertenecen a una circunferencia común. Todos los puntos concíclicos están a la misma distancia del centro del círculo.

Tres puntos en el plano que no caen todos en son concíclicos, pero cuatro o más de esos puntos en el plano no son necesariamente concíclicos.



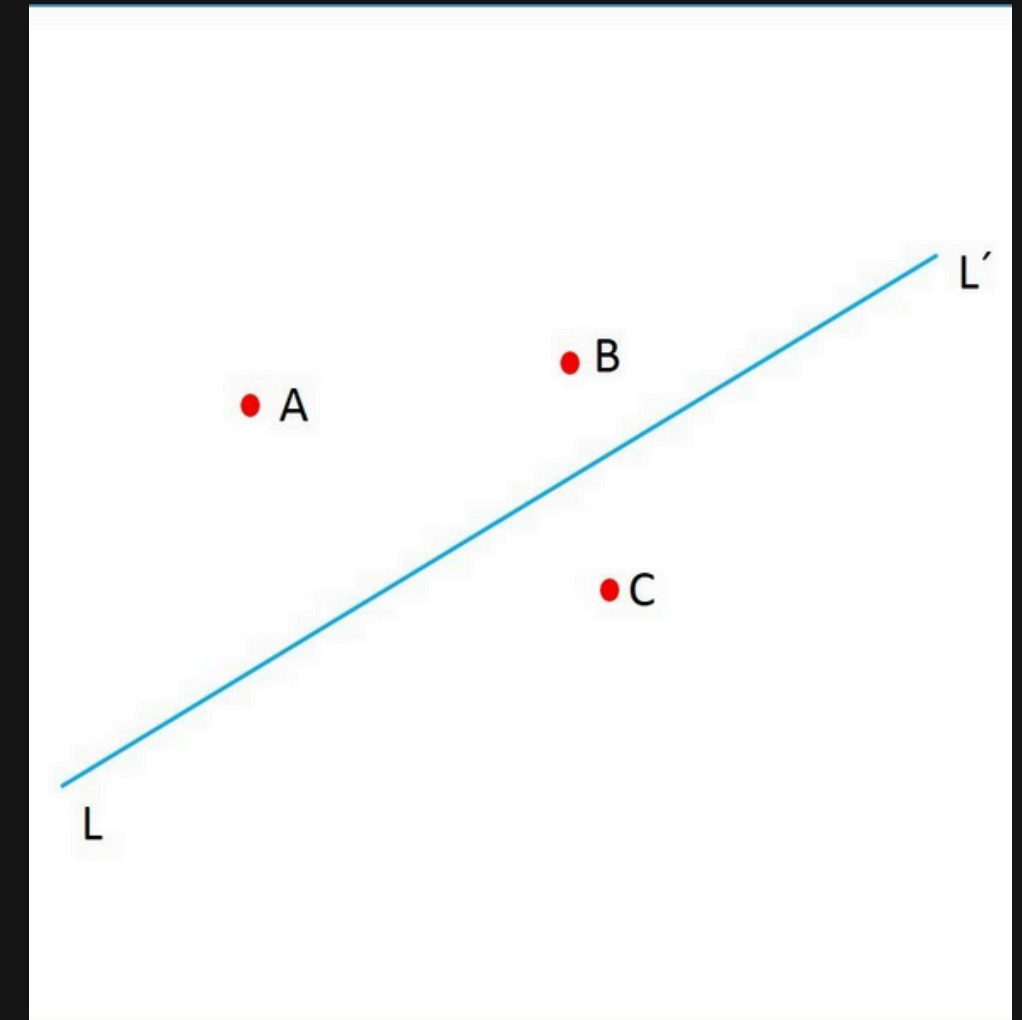
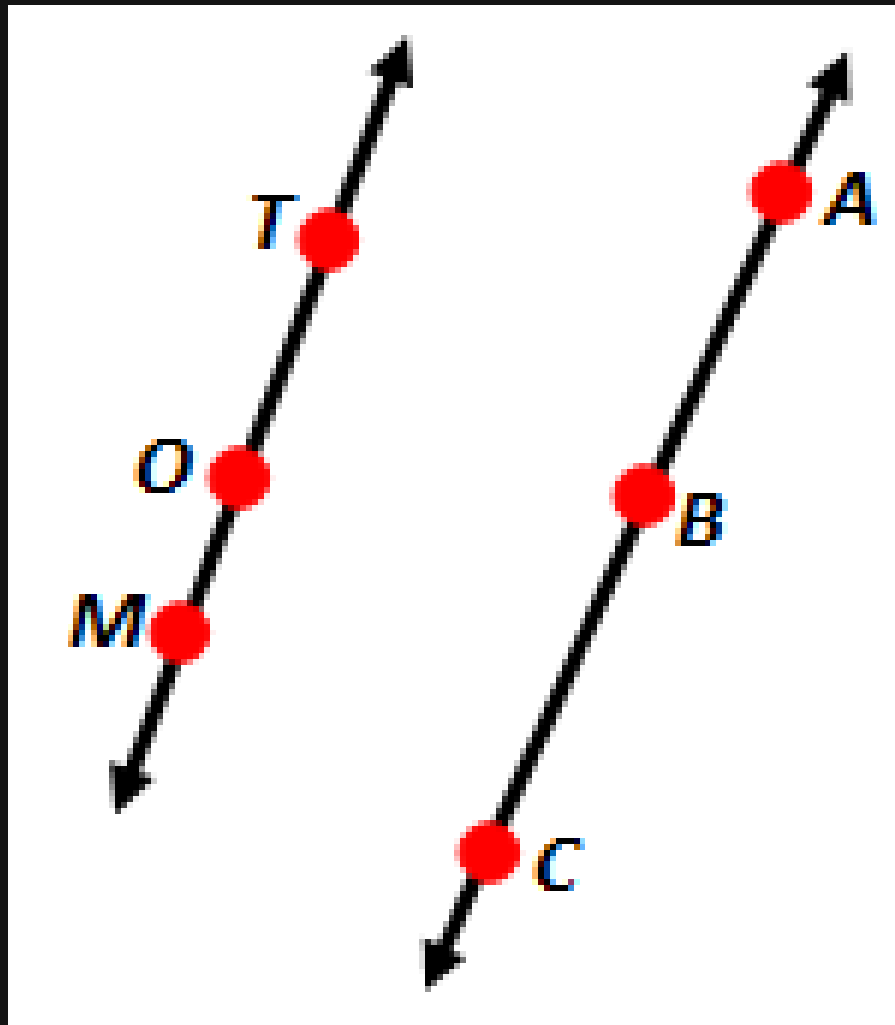
Conceptos previos

Puntos Colindantes

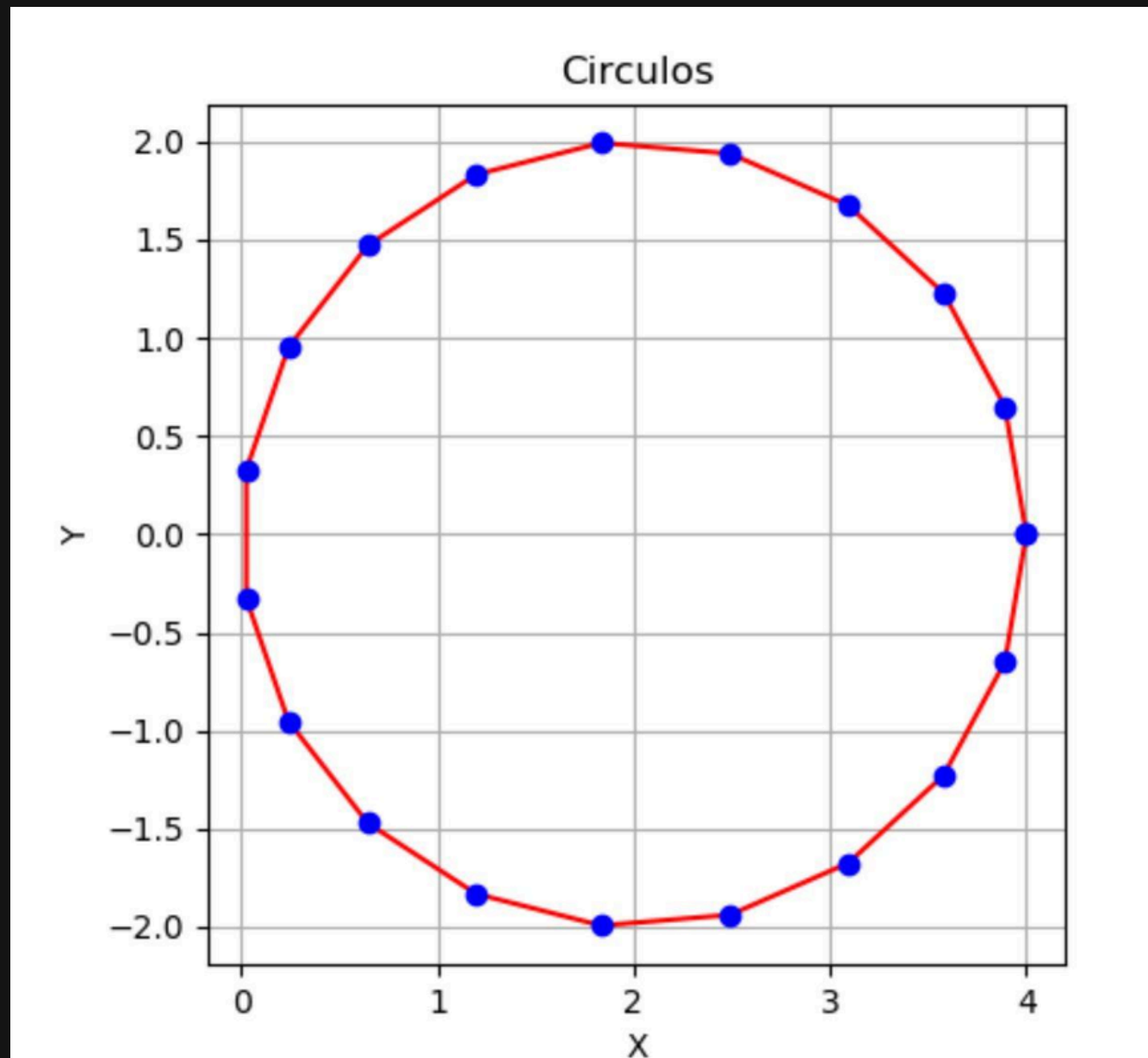
Los puntos colineales son puntos que se encuentran en la misma línea.

La palabra 'colineal' se descompone en el prefijo 'co-' y la palabra 'lineal'.
'Co-' indica unión, como en compañero de trabajo o cooperar.
'Lineal' se refiere a una línea.

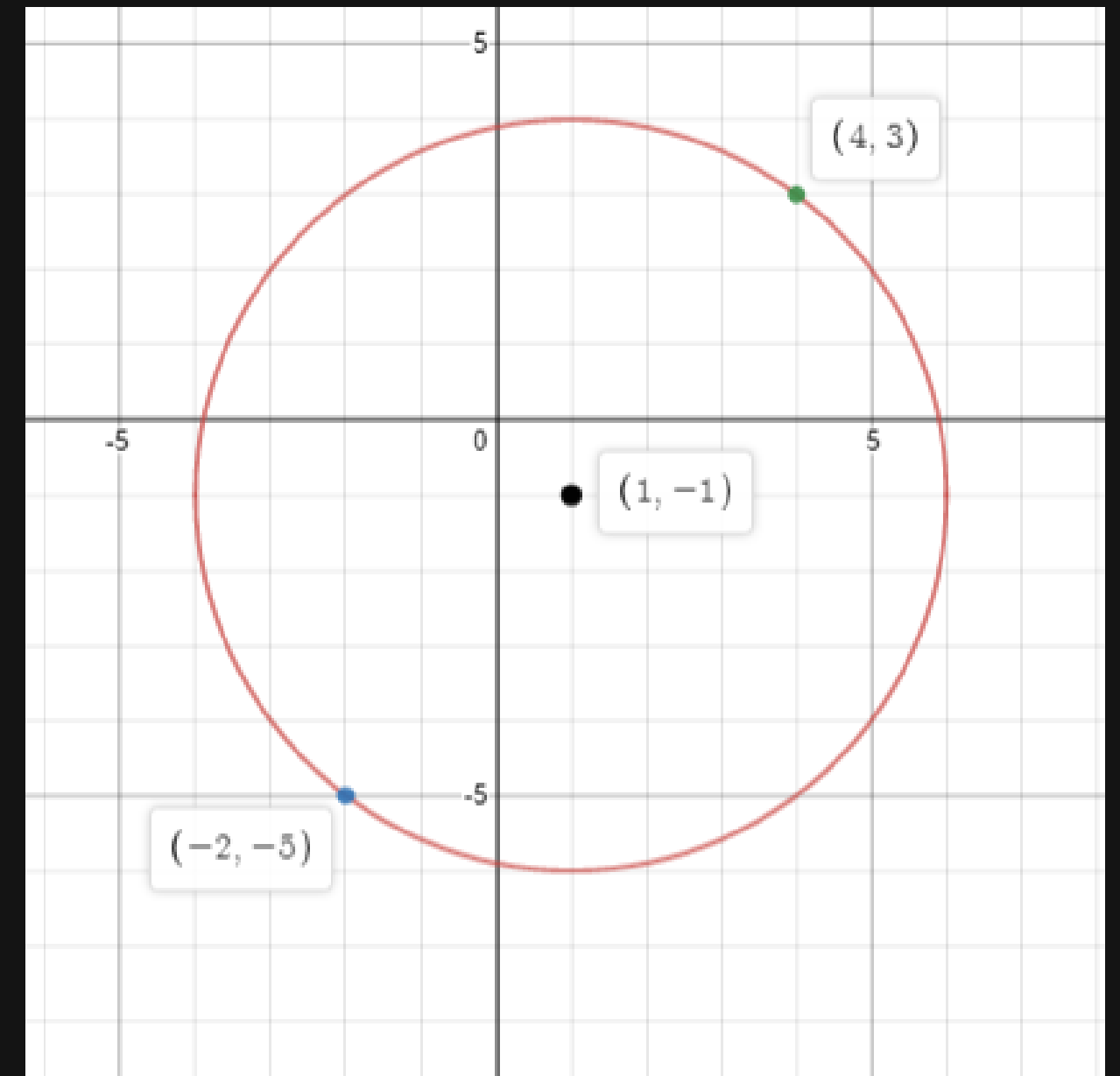
Entonces, colineal básicamente significa puntos que cuelgan juntos en la misma línea.



Conceptos previos



Circunferencia vacía



Circunferencia no vacía

Condicion por delaunay

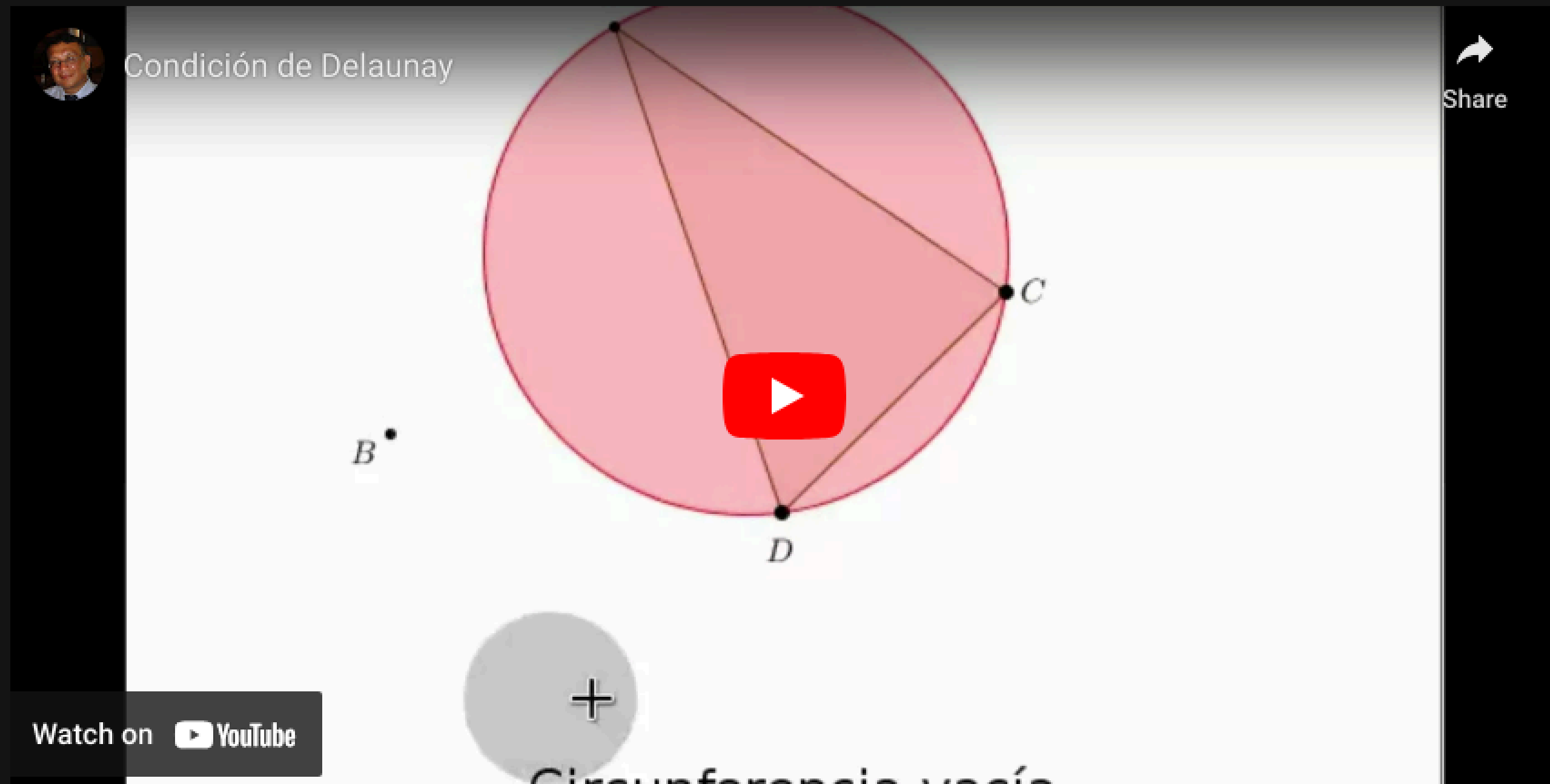
Necesitamos una nube de punto con las siguientes características:

- 1) N puntos no concíclicos
- 2) N puntos no colindantes
- 3) Identificar las circunferencias vacías y no vacías en la nube de puntos

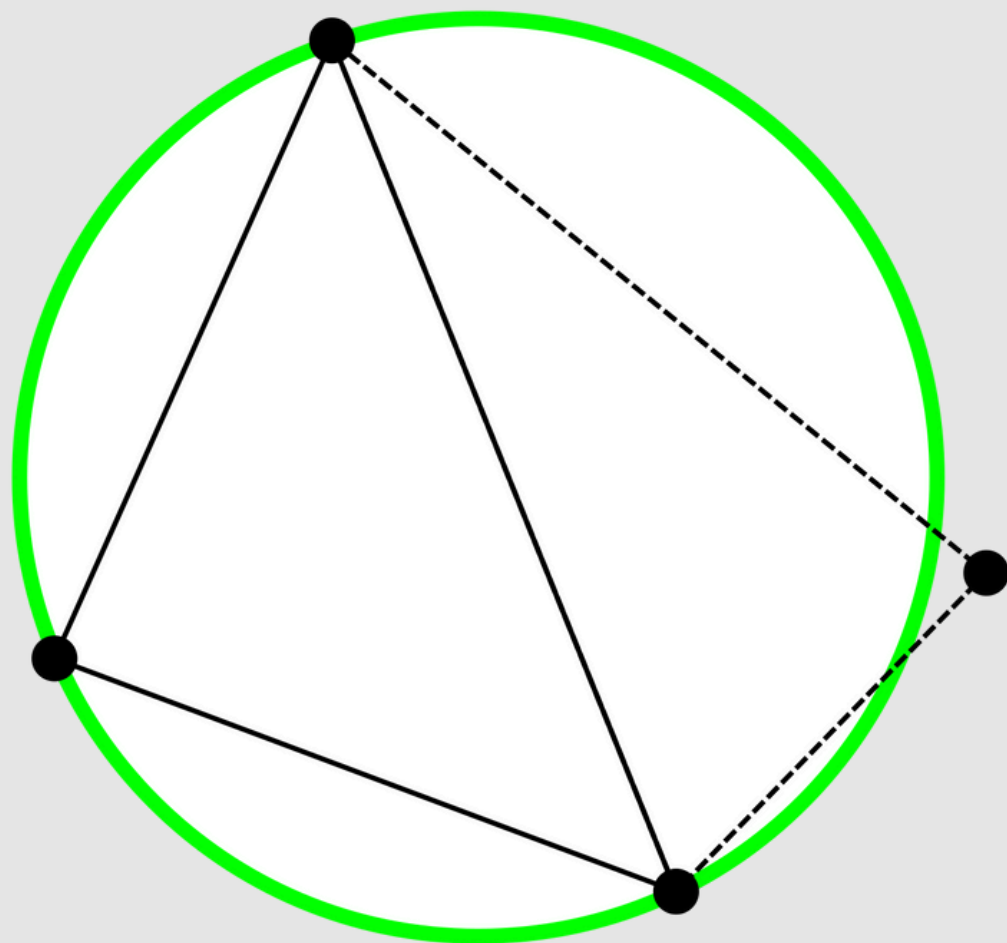
Buscamos determinar triángulos dentro de la nube de puntos cumpliendo que estos subconjuntos de puntos:

- 1) Deben formar una circunferencia vacía para ser valido el triangulo.
- 2) Todos los puntos deben estar conectados
- 3) Los puntos forman una figura geométrica irregular

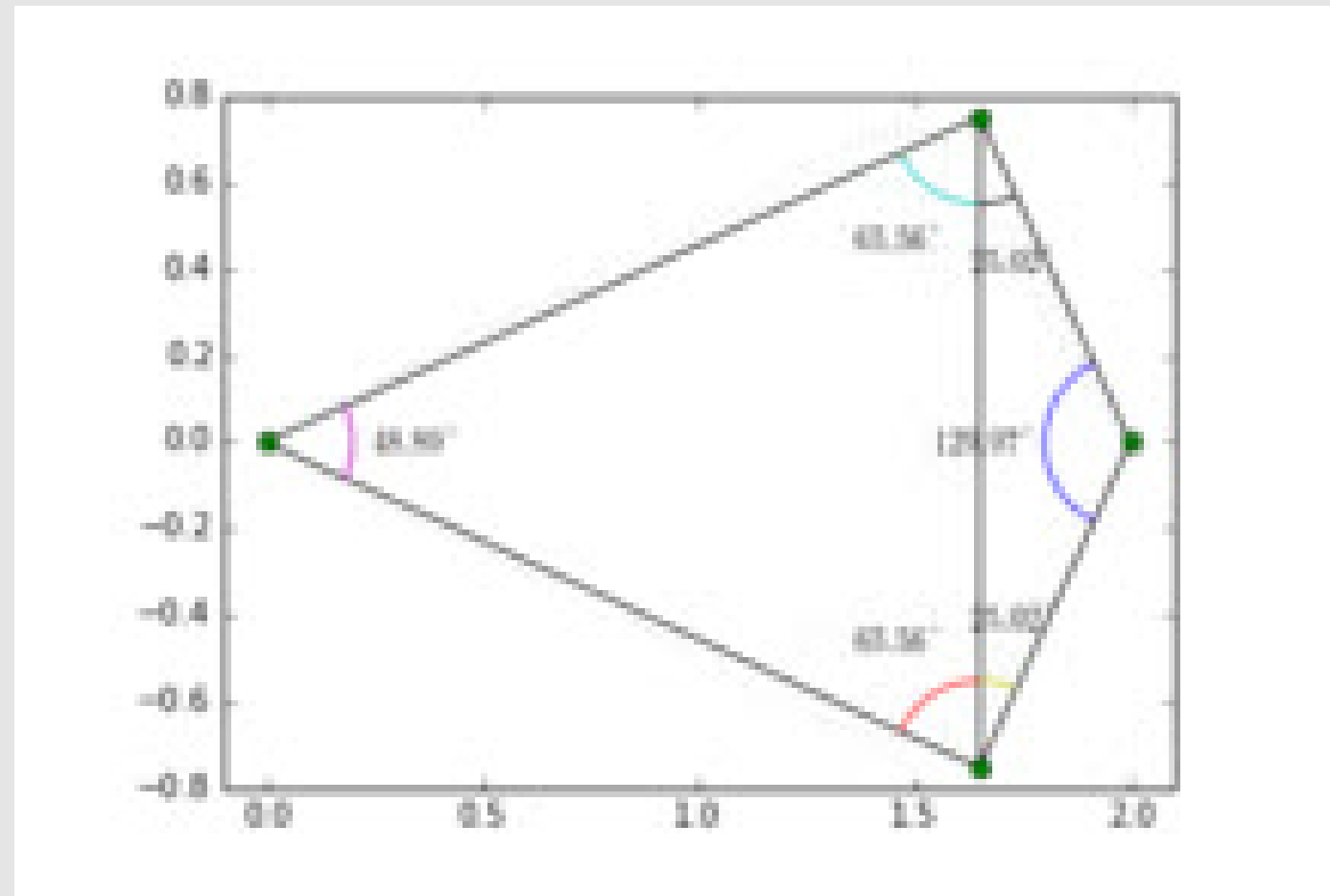
TRIANGULACIÓN DE UNA NUBE DE PUNTOS



Materia



Ejemplo



Pizarra

EJERCICIO

DADO 4 PUNTOS EN UN SISTEMA CARTESIANO $P1=(2,3)$, $P2=(3,5)$, $P3=(-1,7)$, $P4= (-4,-1)$.
USANDO EL ALGORITMO DE TRIANGULACION DE DELAUNAY CALCULE EL AREA DEL
POLIGONO IRREGULAR
ATRAVEZ DE LAS FIGURAS SIMPLES (TRIANGULOS) QUE SE FORMAN LUEGO DE
COMPLIR LOS REQUISITOS.



COMPUTACION GRAFICAS Y SUS
APLICACIONES – COM3201-1

Algoritmos de ráster y Triangulación

COM3201-1
Profesores:
Alfonso Ehijo
Ignacio Bugueño

Ayudante
Alan E. Molina H.
alan.molina@pregrado.uoh.cl